KALASALINGAM ACADEMY OF REEARCH AND EDUCATION

RESEARCH PAPER EXPLAINER ASSISTANT (PAPERLENS)

Project ID: PRJ058

Team ID: T058

Team GitHub Repository: https://github.com/mariselvi242007/research-paper-explainer-assistant

Live Link: https://research-paper-explainer-assistant-pfo2wmn82f9rb3guf8tcv3.streamlit.app/

Team Name: Smart Work

TEAM MEMBERS

Name	Register Number	GitHub Link
M. Mariselvi	99230041094	https://github.com/mariselvi242007/gen-ai-assignments
S. Kavya	99230041093	https://github.com/Kavya-ui-create/Gen-AI.git
Sampathi Yaswanth	99230041039	GitHub - yashu440/GEN-AI-practice-assessment · GitHub
C. Manasa	99230041047	https://github.com/manasa-gen
M. Ganesh	99230041020	https://github.com/ganesh0404

Project Trainer/Guide

Mr. SHIRWIN PRINCE I

ABSTRACT

Research papers hold a lot of useful knowledge, but they are not easy to read. Students and early researchers often struggle with dense terminology, long explanations, mathematical notation and methods they have never seen before. Going through several papers by hand also eats up a great deal of time.

PaperLens is a Generative AI application built to make this reading easier. A user uploads a research paper as a PDF and then asks questions about it in ordinary language. The system follows a Retrieval-Augmented Generation (RAG) design. Text is pulled out of the PDF page by page with `pypdf`, cut into overlapping chunks with LangChain text splitters, turned into vectors with the Sentence Transformers model `all-MiniLM-L6-v2`, and stored in ChromaDB so that the passages closest in meaning to a question can be found quickly.

To write the final answer, PaperLens calls Groq as its main provider whenever it is configured and reachable, and falls back to Google Gemini if that request fails. The application offers a simple and a technical explanation mode, a structured eight-part summary, page numbers for every retrieved passage, and the passages themselves so that a reader can check the answer against the paper. A Streamlit front end handles uploads, questions and the management of several paper conversations.

The aim of the project is to lower the effort needed to understand academic papers and to give readers a quick way into a paper's objective, method, dataset, results and limitations.

Keywords: Generative AI, Natural Language Processing, Retrieval-Augmented Generation, Large Language Models, Semantic Search, Sentence Transformers, ChromaDB, Groq, Google Gemini, Streamlit

TABLE OF CONTENTS

No.	Title	Page
	ABSTRACT	i
1	INTRODUCTION	1
2	EXISTING SYSTEM AND PROPOSED SYSTEM	3
3	TECHNOLOGICAL BACKGROUND	5
4	SYSTEM REQUIREMENTS	7
5	SYSTEM ARCHITECTURE	8
6	METHODOLOGY	10
7	IMPLEMENTATION DETAILS	12
8	FEATURES AND USER INTERFACE	14
9	TESTING AND VALIDATION	16
10	RESULTS AND DISCUSSION	18
12	FUTURE ENHANCEMENTS	20
13	CONCLUSION	22
A	APPENDIX A: SAMPLE INTERACTION	23
B	APPENDIX B: SAMPLE QUESTIONS FOR DEMONSTRATION	24
C	APPENDIX C: REPOSITORY STRUCTURE	25

CHAPTER 1: INTRODUCTION

1.1 Background

Research papers are where new technical and scientific ideas are first written down. A typical paper states a problem, proposes a solution, describes how it was tested, reports numbers and then draws conclusions. For someone new to the field this structure is useful, but the language inside it is often hard going.

Reading a paper well means finding the important sections, understanding the technical parts and linking facts that sit far apart in the document. The job gets harder when the paper is long or when it leans on unfamiliar algorithms and equations.

Large language models can now answer questions and explain ideas in fluent English. The trouble is that a general model does not know what a particular paper actually says. It may give an answer that sounds right but is not supported by the document in front of the reader.

PaperLens tries to close this gap by joining document retrieval with text generation. Rather than trusting only what the model already knows, the application first looks up the passages of the uploaded paper that match the question and gives them to the model as context. It also shows these passages with their page numbers, so the user can find the original text and judge the answer personally.

1.2 Problem Statement

Students and researchers often find it hard to spot a paper's objective, understand its method, locate the dataset, read the results and see the limitations. Existing chat-style AI assistants are not tied to one uploaded document. They can state things the paper never said, and they usually cannot point to the exact place an answer came from.

What is needed is a tool built around a single document. It should retrieve the relevant parts of an uploaded paper, explain them in language the reader can follow, produce a structured summary and show the page evidence behind each answer.

1.3 Objectives

The project has the following objectives:

1. Build a web application that accepts research papers in PDF format.
2. Extract readable text from the PDF while keeping track of page numbers.
3. Split the text into manageable, overlapping chunks.
4. Create semantic embeddings for every chunk.
5. Store the embeddings in ChromaDB and retrieve relevant passages from it.
6. Answer natural-language questions with the RAG approach.
7. Connect Groq and Google Gemini for answer generation.
8. Offer simple and technical explanation modes.
9. Produce structured summaries of research papers.
10. Show source page numbers and retrieved passages for verification.
11. Support several paper conversations and basic chat management.
12. Handle provider errors and quota limits as gracefully as possible.

1.4 Scope

PaperLens is meant for students, researchers, teaching staff and technical professionals who need help understanding research documents. The current version covers PDF upload and text extraction, page-aware chunking, embedding generation, vector retrieval, context-based question answering, simple and technical explanations, structured summaries, source-page references, multiple conversations and provider fallback.

The system works on text-based PDFs. OCR for scanned papers, permanent user accounts and the interpretation of figures and tables are treated as future work.

1.5 Motivation

Many students have to read research papers for final-year projects, literature surveys, seminars and presentations. They usually understand the general topic but get stuck on specialised terms and long passages. We wanted a tool that lets them ask a question in their own words and get an answer pitched at their level. We also wanted the tool to push users toward checking the source instead of simply trusting a generated reply.

CHAPTER 2: EXISTING SYSTEM AND PROPOSED SYSTEM

2.1 Existing System

At present, a paper is analysed either by reading it manually or with general-purpose document and AI tools. Manual reading puts the whole burden of finding key sections and interpreting technical detail on the reader. General AI assistants can explain a concept well, but their answers are not always tied to the text of the uploaded paper. Some reading tools offer summaries or keyword search, yet few of them combine semantic retrieval, question answering, different explanation levels and page-level evidence in one workflow.

2.2 Limitations of the Existing System

1. Manual reading takes a long time.
2. Technical vocabulary is hard for beginners.
3. Keyword search misses passages that use different wording.
4. General language models can produce unsupported statements.
5. It is difficult to find the exact source of an answer.
6. Summaries may leave out important details.
7. Several separate tools may be needed for reading, question answering and summarising.

2.3 Proposed System

PaperLens is a web-based assistant that combines document processing, semantic search and generative AI. The user uploads a paper, and the application extracts its text page by page, splits it into overlapping chunks and converts each chunk into an embedding. The embeddings, together with the text and page metadata, are saved in ChromaDB.

When a question is asked, the system retrieves up to five relevant chunks and hands them to the language model as evidence. Groq is the preferred provider when it is configured and working. If the Groq request fails, Gemini is tried according to the

provider settings. The answer appears together with the retrieved passages and their page numbers, and the user can choose Simple Mode or Technical Mode. The system also builds structured summaries and manages conversations.

2.4 Advantages of the Proposed System

- It cuts down the effort of understanding long papers.
- It lets users talk to a paper in natural language.
- It relies on meaning rather than exact keywords.
- It gives page-level source references.
- It offers beginner-friendly and technical explanations.
- It creates structured summaries.
- It works with two AI providers and can fall back to the second one.
- It supports several paper conversations in one interface.
- It encourages users to verify answers against the paper.

CHAPTER 3: TECHNOLOGICAL BACKGROUND

3.1 Natural Language Processing and Generative AI

Natural Language Processing is the branch of AI that deals with letting computers analyse and produce human language. It covers classification, retrieval, translation, summarisation and question answering. Generative AI refers to systems that create new content such as text, images, audio and code. In PaperLens, NLP techniques match a question to relevant passages, and generative models explain those passages and write summaries. The quality of a response depends on what was retrieved, on the model and on the prompt.

3. 2 Large Language Models in the Project

Groq-hosted language models and Google Gemini models supply the generative ability. Groq is an inference platform and Gemini is Google's model family; the application reaches both through their APIs.

3. 3 Text Embeddings

An embedding represents a word, sentence or passage as a vector of numbers. Passages with similar meaning get nearby vectors even if they share few words. PaperLens uses `all-MiniLM-L6-v2` for both document chunks and user questions.

3. 4 Vector Database

A vector database stores embeddings and returns the nearest ones for a query. PaperLens uses ChromaDB to store chunk text, embeddings and page numbers. The implementation uses an in-memory client, so stored vectors are not guaranteed to survive after the application stops.

3.5 PDF Text Extraction

The `pypdf` library extracts text from each page, which lets the application remember the page number of every piece of text. Quality depends on how the PDF was created, and scanned documents need OCR first.

3.6 Text Chunking

Long documents are divided into smaller pieces before embedding. PaperLens uses LangChain's `RecursiveCharacterTextSplitter` with the settings below. Overlap keeps context intact when an explanation crosses a chunk boundary.

Parameter	Value
Chunk size	1,000 characters
Chunk overlap	200 characters
Retrieval limit	Up to 5 chunks

CHAPTER 4: SYSTEM REQUIREMENTS

4.1 Hardware Requirements

Component	Requirement
Processor	Modern dual-core or better
RAM	8 GB recommended
Storage	Enough space for Python packages and documents
Internet	Needed for the AI APIs and for installing packages
Display	Standard monitor or laptop screen

4.2 Software Requirements

Component	Technology
Programming language	Python
Interface	Streamlit
PDF processing	pypdf
Text splitting	LangChain Text Splitters
Embedding library and model	Sentence Transformers, all-MiniLM-L6-v2
Vector database	ChromaDB
Primary AI provider	Groq
Fallback AI provider	Google Gemini
Architecture	Retrieval-Augmented Generation
Development environment	Google Colab and/or a local Python environment
Version control	Git and GitHub
Optional deployment	Streamlit Community Cloud

CHAPTER 5: SYSTEM ARCHITECTURE

5.1 Architecture Overview

PaperLens is made of components for document processing, embedding, vector storage, retrieval, answer generation and the user interface. Figure 5.1 shows the flow of data.

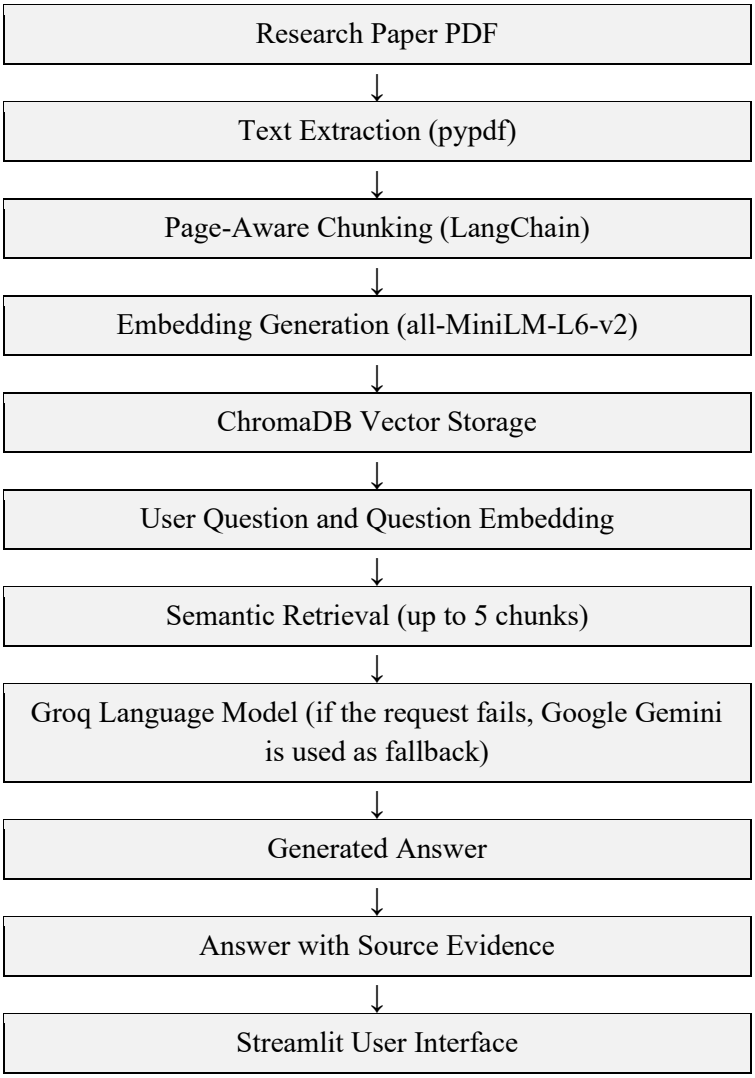


Figure 5.1: Overall architecture of PaperLens

Gemini acts as the fallback when Groq is configured as the primary provider and fails. If no Groq key is configured, the application can use Gemini directly.

Summarisation follows a separate path in which the extracted paper text is sent to the configured provider. How well it works depends on the summarisation function and on the model's context and output limits.

5.2 Main Modules

Module 1: User interface. The Streamlit interface lets users upload PDFs, type questions, choose an explanation mode, read answers, inspect evidence and manage conversations.

Module 2: PDF processing. Extracts readable text from each page and keeps the page number.

Module 3: Text chunking. Splits the text into smaller overlapping chunks that are easy to embed and retrieve.

Module 4: Embedding generation. Sentence Transformers turns each chunk into a vector. The same model embeds the user's question so both live in the same vector space.

Module 5: Vector storage. ChromaDB keeps embeddings together with chunk text and page metadata.

Module 6: Semantic retrieval. The question embedding is compared with the stored vectors and up to five relevant chunks are returned.

Module 7: AI response generation. The question and retrieved chunks go to the language model, with a prompt that tells it to answer from the paper's context. Groq is preferred and Gemini is the fallback.

Module 8: Source evidence. Retrieved passages and page numbers are shown so users can find the original text. They support the answer but do not guarantee that every sentence is fully backed by the source.

Module 9: Summary generation. Produces a structured summary covering objective, problem, methodology, dataset, technologies, results, limitations and conclusion.

Module 10: Conversation management. Supports several paper conversations, recent chats, pinned chats, clearing messages and deleting chats. Whether this data survives depends on how it is stored; session-state data can be lost when the session ends.

CHAPTER 6: METHODOLOGY

6.1 Uploading the Paper

The user picks a PDF through the Streamlit interface, and the application starts processing its text.

6.2 Extracting Text

`pypdf` reads the document one page at a time. Keeping the page number with each piece of text is what later allows retrieved passages to be traced back to their location. For example, page 1 might hold the abstract and introduction, page 3 the proposed method and page 4 the experiments, although the real layout depends on the uploaded paper.

6.3 Splitting the Document

`RecursiveCharacterTextSplitter` divides the text into chunks of 1,000 characters with 200 characters of overlap. The overlap helps when an explanation runs across a chunk boundary.

6.4 Generating Embeddings

Each chunk is converted to a vector by `all-MiniLM-L6-v2`. As an example, "The proposed system detects malicious websites" and "The method identifies potentially harmful web pages" use different words but should end up with related vectors. This is what allows conceptually similar passages to be found [12].

6.5 Storing Embeddings

ChromaDB stores each chunk's text, its embedding and its page number. This becomes the searchable form of the uploaded paper. In the current configuration the storage is in memory and only lasts for the running process.

6.6 Processing the Question

When the user asks something like "What problem does this paper solve?", the same embedding model converts the question into a vector. Using one model for both sides keeps the comparison meaningful.

6.7 Retrieving Passages

ChromaDB compares the question vector with the stored vectors and returns up to five relevant chunks. Retrieval quality depends on the embedding model, the document structure, the chunk settings and the wording of the question. Passing only a few chunks also fits the observation that models handle long, crowded contexts less reliably [18].

6.8 Generating the Answer

The question and the retrieved passages are sent to the selected provider, and the model is told to answer from that material. Provider selection works as follows:

1. Use Groq if it is configured as the primary provider.
2. If the Groq request fails, try Gemini according to the fallback logic.
3. If Groq is not configured, use Gemini directly when available.
4. If no provider can respond, pass the problem to the error-handling code.

The exact models and retry behaviour depend on the implementation and on what the providers offer at the time.

6.9 Displaying the Answer and Sources

The answer is shown in the Streamlit interface along with the retrieved passages and their page numbers. Users can compare these with the original paper. The sources show what evidence was retrieved, but important claims should still be checked directly in the paper [20].

6.10 Generating the Paper Summary

The summarisation function sends the extracted paper text to the language model and asks for a structured summary with these sections: Research Objective, Problem Statement, Methodology, Dataset, Algorithms and Technologies Used, Main Results, Limitations and Conclusion. If the paper does not contain a requested item, the model is told to write "Not specified in the paper." The summary can be incomplete when text extraction misses content or when the paper is longer than the model can handle [26].

CHAPTER 7: IMPLEMENTATION DETAILS

7.1 Python

Python carries the whole application: PDF handling, chunking, embeddings, vector storage, retrieval and calls to the AI APIs. Its library ecosystem for NLP, document processing and web apps made it a natural choice.

7.2 Streamlit

Streamlit builds the web interface with upload widgets, text inputs, buttons, chat messages and summary displays, and no separate front-end framework is needed.

7.3 PDF Processing with pypdf

The application reads each page with `pypdf` and extracts its text, which preserves page metadata. The library cannot recognise text inside scanned images, so image-only PDFs would need OCR.

7.4 Text Splitting with LangChain

`RecursiveCharacterTextSplitter` is configured with a chunk size of 1,000 characters and an overlap of 200 characters. These values try to balance context against retrieval efficiency and may need tuning for other kinds of documents.

7.5 Sentence Transformers

The `all-MiniLM-L6-v2` model produces the embeddings for chunks and questions, in line with the sentence-embedding approach of [12] and the compact architecture of [13].

7.6 ChromaDB

ChromaDB stores chunk embeddings, text and metadata and returns relevant passages for a query. The in-memory client used here does not keep data after a restart.

7.7 Groq API

Groq provides API access to supported language models and is the preferred generation provider when configured. The chosen model must be available to the account at run time, and both availability and quotas can change.

7.8 Google Gemini API

Gemini [5] is the alternative provider. It is attempted when the Groq request fails and used directly when no Groq key exists. The model names and retry order must match what the API supports at deployment time.

7.9 Prompt Design

The prompt steers the model toward answers based on the supplied passages. In concept it says: answer the question using the provided research-paper context; if the context is not enough, say that the answer is not specified in the provided content; do not invent results or dataset details; and explain in the selected Simple or Technical mode. The real prompt may include more rules for formatting and sources. Prompting a model in this way relies on the in-context ability described in [3].

7.10 API Key Management

Valid keys are needed for the providers in use. For local runs they can be placed in Streamlit secrets or in environment variables. A sample secrets file looks like this:

```
GEMINI_API_KEY = "your_gemini_api_key"
GROQ_API_KEY = "your_groq_api_key"
```

These are placeholders and not real credentials. The secrets file must never be committed to a public repository.

CHAPTER 8: FEATURES AND USER INTERFACE

8.1 PDF Upload

Users upload papers as PDFs, and the system processes them to prepare for retrieval and question answering.

8.2 Natural-Language Question Answering

Users can ask in everyday language instead of guessing keywords. Typical questions include: What problem does this paper solve? What methodology is used? Which dataset is used? Which algorithms are proposed? What are the main findings? What limitations are mentioned? Relevant passages are retrieved before an answer is written.

8.3 Simple Explanation Mode

Simple Mode uses plain English and fewer technical terms. It suits students meeting a topic for the first time or preparing for a project review.

8.4 Technical Explanation Mode

Technical Mode gives more detailed answers with research terminology and computer-science concepts. It suits methodology analysis and technical discussions.

8.5 Structured Summary

Section	Purpose
Research Objective	What the research aims to achieve
Problem Statement	The problem being addressed
Methodology	The proposed approach
Dataset	The data used, if stated
Algorithms and Technologies	Methods and tools described
Main Results	Reported findings
Limitations	Stated weaknesses or constraints
Conclusion	The paper's final observations

8.6 Source-Page References

Retrieved passages appear with page numbers so that users can verify the response. This is more traceable than an answer with no document evidence.

8.7 Multiple Paper Conversations

Users can keep separate conversations for different papers, with recent chats, pinned chats, message clearing and chat deletion. How long these last and how well they are separated depends on the implementation.

8.8 AI Provider Fallback

PaperLens supports Groq and Gemini. When the first request fails, the application tries the other provider. This cannot guarantee a reply if both services are down, the keys are invalid or the quotas are used up.

CHAPTER 9: TESTING AND VALIDATION

9.1 Testing Objective

Testing checks whether PaperLens performs document processing and question answering correctly. It should use representative papers, questions with known answers, questions whose answers are not in the paper and different provider-availability situations. The table below is a functional test plan; actual outcomes must be recorded after running the final application.

9.2 Functional Test Cases

Test ID	Scenario	Expected Result
TC01	Upload a valid text-based PDF	The PDF is accepted and processed
TC02	Upload an invalid or unsupported file	The error is handled properly
TC03	Extract text from several pages	Readable text with page information
TC04	Split a long document	Overlapping chunks are produced
TC05	Generate embeddings	Chunks become vectors
TC06	Store document vectors	Chunks and metadata are saved in ChromaDB
TC07	Ask a question answered in the paper	Relevant passages retrieved and a context-based answer generated
TC08	Ask using different wording	Semantically relevant passages retrieved when available
TC09	Ask a question the paper does not answer	The application says sufficient information is not available
TC10	Select Simple Mode	Answer uses beginner-friendly language
TC11	Select Technical Mode	Answer is more technical
TC12	Request a summary	Structured sections generated where text and model limits allow
TC13	Inspect source references	Passages and page numbers are shown
TC14	Simulate a Groq failure	Gemini is attempted under the fallback logic
TC15	Run without a Groq key	Gemini is used directly when configured

Test ID	Scenario	Expected Result
TC16	Exceed an API quota	The error is handled and not shown as a success
TC17	Use chat-management controls	Each operation does what it should
TC18	Restart the application	In-memory vectors and session chats behave as documented

9.3 Testing Categories

Unit testing checks single functions such as PDF extraction, chunking, metadata creation and provider selection.

Integration testing checks that PDF processing, embedding, ChromaDB retrieval and answer generation work together.

Functional testing checks that the interface performs the expected actions.

Error-handling testing covers missing API keys, invalid documents, network failures, unsupported models and quota errors.

Answer-quality testing compares generated answers with the paper to find unsupported claims, missing information and retrieval failures.

9.4 Evaluation Metrics

The following metrics can be used in a later formal evaluation, drawing on ideas from [20] and [21].

Metric	Purpose
Retrieval Precision@k	Share of retrieved passages judged relevant
Retrieval Recall@k	How much of the relevant evidence is retrieved
Answer Correctness	Agreement with verified reference answers
Faithfulness	Whether claims are supported by the supplied evidence
Response Latency	Time taken to return an answer
Summary Completeness	Whether the important paper sections are covered
Error Rate	Proportion of failed requests or operations

CHAPTER 10: RESULTS AND DISCUSSION

10.1 Expected Functional Outcome

PaperLens is designed as a single interface where a user can upload a paper, ask questions, see the retrieved passages and generate a structured summary. The intended workflow is:

1. Upload a research paper in PDF format.
2. Process its text into searchable chunks.
3. Ask natural-language questions.
4. Retrieve relevant passages through semantic similarity.
5. Generate an answer with Groq or Gemini.
6. Inspect the retrieved passages and page references.
7. Switch between Simple and Technical modes.
8. Generate a structured summary.
9. Manage several paper conversations.

These are the intended behaviours of the implementation described in this report. A final demonstration and test log should confirm that each one works in the submitted version.

10.2 Discussion

The main strength of the RAG approach is that the generation step receives passages from the paper itself and does not rely only on what the model remembers [6], [10]. Semantic retrieval can find related content even when a question is worded differently from the paper [7], [12], and page metadata helps users return to the source.

The two explanation modes let the same system serve beginners and more advanced readers, and the structured summary gives a quick view of the objective, method, dataset, results and limitations.

There are also clear risks. If the right passage is not retrieved, the answer may be incomplete. A model can misread its evidence or add claims that the passages do not support [19], [20]. Page references therefore make an answer easier to check, but they are not proof that it is correct.

10.3 Quantitative Results

No verified benchmark figures, timing measurements or formal answer-quality scores are available for this project. For that reason no numbers on accuracy, precision, recall or speed are reported. Such results can be added later by running the application on a suitable set of papers and questions and measuring retrieval relevance, answer correctness, faithfulness and latency, for example with the approach of [21].

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 OCR Support

Add optical character recognition so that scanned and image-based papers can be processed.

11.2 Persistent Vector Storage

Use persistent ChromaDB storage or another vector database so uploaded papers can be reused after a restart. Larger collections could use an indexing library such as FAISS [17].

11.3 Persistent Chat History

Add a database-backed chat system with user accounts.

11.4 Multi-Paper Comparison

Allow several papers to be uploaded and compared on objectives, methods, datasets, results and limitations.

11.5 Claim-Level Citations

Link each generated claim to a specific retrieved passage and page [20].

11.6 Hybrid Search and Re-ranking

Combine semantic retrieval with keyword search such as BM25 [14] and add a re-ranking stage [15], [16] to improve the relevance of evidence.

11.7 Adaptive Retrieval

Explore models that decide when to retrieve and critique their own output [11].

11.8 Figure and Table Understanding

Extract and interpret charts, equations and tables that carry important research information.

11.9 Summary Export and Multilingual Support

Let users export structured summaries as PDF or other formats, and offer explanations in several languages.

11.10 Formal Evaluation

Build a benchmark of papers and verified questions, in the spirit of [21] and [25], to measure retrieval quality, answer correctness, grounding, latency and user satisfaction.

CHAPTER 14: CONCLUSION

PaperLens is a Generative AI application that makes research papers easier to understand through natural-language interaction. It joins PDF text extraction, overlapping chunking, semantic embeddings, vector storage and Retrieval-Augmented Generation to retrieve relevant passages and write answers from them. Embeddings come from the Sentence Transformers model `all-MiniLM-L6-v2`, retrieval is handled by ChromaDB, Groq is the preferred generation provider, and Gemini serves as the alternative.

The Streamlit application offers simple and technical explanations, structured summaries, page references, retrieved evidence and conversation management. By combining retrieval with generation, it reduces the effort of exploring technical documents, and its source evidence encourages users to check answers instead of trusting them blindly.

The literature reviewed in Chapter 3 shows that each building block of the system is well supported by research, while also showing that hallucination, weak attribution and long-context behaviour remain open problems. Future work on OCR, persistent storage, multi-paper comparison, claim-level citations, hybrid retrieval and systematic evaluation could make the tool more reliable for wider academic use.

APPENDIX A: SAMPLE INTERACTION

User question: What is the main objective of this research paper?

Processing steps:

1. The question is converted into an embedding.
2. Up to five relevant chunks are retrieved from ChromaDB.
3. The chunks are sent to the configured AI provider.
4. An answer is generated in the selected explanation mode.
5. The answer is displayed with the retrieved passages and page references.

Illustrative answer format: "The main objective of this research is to [objective stated in the uploaded paper]. The authors aim to address [problem described in the paper] using [proposed method]." The placeholders must be replaced with information actually retrieved from the uploaded paper.

APPENDIX B: SAMPLE QUESTIONS FOR DEMONSTRATION

1. What is the main objective of the paper?
2. Explain the problem statement in simple English.
3. What methodology did the authors use?
4. Which dataset was used in the study?
5. Explain the proposed algorithm step by step.
6. What are the main experimental results?
7. What limitations are mentioned in the paper?
8. What is the main contribution of the research?
9. What future work do the authors suggest?
10. Summarise the paper using the eight structured sections.

APPENDIX C: REPOSITORY STRUCTURE

This is a suggested layout. Keep only the files and folders that really exist in the submitted repository.

```
research_paper_explainer-assistant/
|-- app.py
|-- requirements.txt
|-- README.md
|-- .gitignore
|-- notebooks/
|   |-- research_paper_explainer.ipynb
|-- screenshots/
|-- .streamlit/
    |-- secrets.toml
```

The `secrets.toml` file is for local use only and must not be committed to a public repository.